

计算几何算法专题

算法学习笔记

2026 年 5 月 9 日

目录

1	基础运算	3
1.1	点积与叉积	3
1.2	点到直线距离	4
1.3	点到线段距离	5
2	线段相交	6
2.1	判断线段相交（规范相交）	6
2.2	判断线段相交（包括端点）	7
2.3	求线段交点坐标	9
3	凸包	10
3.1	Andrew 算法求凸包	10
3.2	判断点是否在凸多边形内	12
3.3	判断点是否在任意多边形内（射线法）	13
4	多边形面积	14
4.1	计算多边形面积（鞋带公式）	14
4.2	判断多边形方向（顺时针/逆时针）	15
5	最近点对	16
5.1	分治法求最近点对距离	16
6	旋转卡壳	18
6.1	凸包直径（最远点对）	18
6.2	凸包最小宽度	19
7	半平面交	20
7.1	半平面交求可行域	20

8	圆的相关计算	23
8.1	两圆交点	23
8.2	点到圆的切线	24
8.3	圆与直线交点	26
9	最小圆覆盖	27
9.1	随机增量法求最小圆覆盖	27
10	计算几何总结	29

1 基础运算

1.1 点积与叉积

解决问题：点积用于判断向量夹角（锐角/钝角/直角），叉积用于判断方向（顺时针/逆时针/共线）和计算面积。适用于所有几何问题的底层计算。

核心概念

给定向量 $\vec{a} = (x_1, y_1)$, $\vec{b} = (x_2, y_2)$:

- 点积: $\vec{a} \cdot \vec{b} = x_1x_2 + y_1y_2 = |\vec{a}||\vec{b}| \cos \theta$
- 叉积: $\vec{a} \times \vec{b} = x_1y_2 - x_2y_1 = |\vec{a}||\vec{b}| \sin \theta$

叉积的正负：正表示 $\vec{b}$ 在 $\vec{a}$ 的逆时针方向，负表示顺时针方向，0 表示共线。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      // 定义点结构体
4      struct Point {
5          double x, y; // x: 点的横坐标, y: 点的纵坐标
6          Point(double x = 0, double y = 0) : x(x), y(y) {}
7      };
8      // 定义向量为点的别名
9      typedef Point Vector;
10     // 向量加法
11     Vector operator+(Vector a, Vector b) {
12         return Vector(a.x + b.x, a.y + b.y); // 对应坐标相加
13     }
14     // 向量减法
15     Vector operator-(Vector a, Vector b) {
16         return Vector(a.x - b.x, a.y - b.y); // 对应坐标相减
17     }
18     // 向量数乘
19     Vector operator*(Vector a, double k) {
20         return Vector(a.x * k, a.y * k); // 每个坐标乘以标量k
21     }
22     // 点积:  $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}|\cos$ 
23     double dot(Vector a, Vector b) {
24         return a.x * b.x + a.y * b.y; // 对应坐标乘积之和
25     }
26     // 叉积:  $\vec{a} \times \vec{b} = |\vec{a}||\vec{b}|\sin$ , 正值表示b在a的逆时针方向

```

```

27     double cross(Vector a, Vector b) {
28         return a.x * b.y - a.y * b.x; // 行列式值
29     }
30     // 向量长度
31     double length(Vector a) {
32         return sqrt(dot(a, a)); // 点积开方
33     }
34     // 两向量夹角（弧度）
35     double angle(Vector a, Vector b) {
36         return acos(dot(a, b) / length(a) / length(b)); // 点积除以模长积
37     }
38     // 向量旋转（逆时针旋转弧度rad）
39     Vector rotate(Vector a, double rad) {
40         return Vector(a.x * cos(rad) - a.y * sin(rad), a.x * sin(rad) + a.y
41             * cos(rad));
42     }
43     int main() {
44         Point p1(1, 0), p2(0, 1); // p1: 点1坐标(1,0), p2: 点2坐标(0,1)
45         Vector v1 = p1, v2 = p2; // v1: 向量1, v2: 向量2
46         cout << "点积: " << dot(v1, v2) << "\n"; // 输出0, 表示垂直
47         cout << "叉积: " << cross(v1, v2) << "\n"; // 输出1, 表示v2在v1逆时针
48             方向
49     }
50 }

```

1.2 点到直线距离

解决问题：给定一点和一条直线（由两点确定），求点到直线的距离。适用于碰撞检测、路径规划等场景。

题目描述

给定点 P 和直线 AB ，求点 P 到直线 AB 的距离。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     typedef Point Vector;

```

```

8     double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
9     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
10    double length(Vector a) { return sqrt(dot(a, a)); }
11    // 点到直线距离: 利用叉积的几何意义 (平行四边形面积/底边长)
12    double pointToLine(Point p, Point a, Point b) {
13        Vector ap = p - a;    // ap: 向量从a指向p
14        Vector ab = b - a;    // ab: 向量从a指向b
15        return fabs(cross(ap, ab)) / length(ab); // 叉积绝对值 = 平行四边形面
           积, 除以底边长得高
16    }
17    int main() {
18        Point p(0, 0); // p: 目标点坐标(0,0)
19        Point a(1, 1); // a: 直线上一点(1,1)
20        Point b(2, 2); // b: 直线上另一点(2,2)
21        double dist = pointToLine(p, a, b); // dist: 点到直线距离
22        cout << fixed << setprecision(2) << dist << "\n"; // 输出0.00 (点在直
           线上)
23        return 0;
24    }

```

1.3 点到线段距离

解决问题: 给定点和一条线段, 求点到线段的最短距离。适用于最近点查找、碰撞检测等场景。

题目描述

给定点 P 和线段 AB , 求点 P 到线段 AB 的最短距离。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     typedef Point Vector;
8     double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
9     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
10    double length(Vector a) { return sqrt(dot(a, a)); }
11    // 点到线段距离: 需要判断投影点是否在线段上
12    double pointToSegment(Point p, Point a, Point b) {

```

```

13     Vector ap = p - a; // ap: 向量从a指向p
14     Vector ab = b - a; // ab: 向量从a指向b
15     Vector bp = p - b; // bp: 向量从b指向p
16     double t = dot(ap, ab) / dot(ab, ab); // t: p在直线AB上的投影参数 (0
        ~1之间表示在线段上)
17     if (t < 0) return length(ap); // 投影在A点左侧, 最近点为A
18     if (t > 1) return length(bp); // 投影在B点右侧, 最近点为B
19     Point projection = a + ab * t; // projection: 投影点坐标
20     return length(p - projection); // 投影点在线段上, 返回点到投影点的距
        离
21 }
22 int main() {
23     Point p(1, 1); // p: 目标点坐标(1,1)
24     Point a(0, 0); // a: 线段端点A(0,0)
25     Point b(2, 0); // b: 线段端点B(2,0)
26     double dist = pointToSegment(p, a, b); // dist: 点到线段距离
27     cout << fixed << setprecision(2) << dist << "\n"; // 输出1.00
28     return 0;
29 }

```

2 线段相交

2.1 判断线段相交（规范相交）

解决问题：判断两条线段是否相交（交点在线段内部，不包括端点）。适用于地图导航、电路布线等场景。

题目描述

给定两条线段 AB 和 CD ，判断它们是否规范相交（交点在内部，非端点）。

核心原理

利用叉积判断跨立：点 A 和 B 在线段 CD 两侧，且点 C 和 D 在线段 AB 两侧。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };

```

```

7     typedef Point Vector;
8     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9     // 判断点是否在线段上（包括端点）
10    bool onSegment(Point p, Point a, Point b) {
11        double minx = min(a.x, b.x), maxx = max(a.x, b.x); // minx: 线段x坐标
12        // 最小值, maxx: 线段x坐标最大值
13        double miny = min(a.y, b.y), maxy = max(a.y, b.y); // miny: 线段y坐标
14        // 最小值, maxy: 线段y坐标最大值
15        return cross(p - a, p - b) == 0 && p.x >= minx && p.x <= maxx && p.y
16            >= miny && p.y <= maxy;
17    }
18    // 判断两线段是否相交（规范相交：交点在线段内部，不包括端点）
19    bool segmentsIntersect(Point a, Point b, Point c, Point d) {
20        double d1 = cross(b - a, c - a); // d1: 向量AB与AC的叉积
21        double d2 = cross(b - a, d - a); // d2: 向量AB与AD的叉积
22        double d3 = cross(d - c, a - c); // d3: 向量CD与CA的叉积
23        double d4 = cross(d - c, b - c); // d4: 向量CD与CB的叉积
24        // 规范相交：两线段互相跨立（叉积异号）
25        if (d1 * d2 < 0 && d3 * d4 < 0) return true;
26        // 非规范相交（端点相交）返回false，本函数只判断规范相交
27        return false;
28    }
29    int main() {
30        Point a(0, 0), b(2, 2); // a,b: 第一条线段端点
31        Point c(0, 2), d(2, 0); // c,d: 第二条线段端点
32        if (segmentsIntersect(a, b, c, d)) {
33            cout << "相交\n"; // 输出交集
34        } else {
35            cout << "不相交\n";
36        }
37        return 0;
38    }

```

2.2 判断线段相交（包括端点）

解决问题：判断两条线段是否相交（包括交点在端点的情况）。适用于多边形切割、图形学等场景。

题目描述

给定两条线段 AB 和 CD ，判断它们是否相交（包括端点在对方线段上的情况）。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      typedef Point Vector;
8      double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9      double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
10     // 判断点是否在线段上（包括端点）
11     bool onSegment(Point p, Point a, Point b) {
12         double minx = min(a.x, b.x), maxx = max(a.x, b.x); // minx: 线段x最小
13         // 值, maxx: 线段x最大值
14         double miny = min(a.y, b.y), maxy = max(a.y, b.y); // miny: 线段y最小
15         // 值, maxy: 线段y最大值
16         return cross(p - a, p - b) == 0 && p.x >= minx && p.x <= maxx && p.y
17             >= miny && p.y <= maxy;
18     }
19     // 判断两线段是否相交（包括端点在线上）
20     bool segmentsIntersectIncludeEnds(Point a, Point b, Point c, Point d) {
21         double d1 = cross(b - a, c - a); // d1: AB与AC叉积
22         double d2 = cross(b - a, d - a); // d2: AB与AD叉积
23         double d3 = cross(d - c, a - c); // d3: CD与CA叉积
24         double d4 = cross(d - c, b - c); // d4: CD与CB叉积
25         // 规范相交: 互相跨立
26         if (d1 * d2 < 0 && d3 * d4 < 0) return true;
27         // 端点在线段上
28         if (onSegment(c, a, b)) return true;
29         if (onSegment(d, a, b)) return true;
30         if (onSegment(a, c, d)) return true;
31         if (onSegment(b, c, d)) return true;
32         return false;
33     }
34     int main() {
35         Point a(0, 0), b(2, 0); // a,b: 第一条线段端点
36         Point c(1, 0), d(3, 0); // c,d: 第二条线段端点
37         if (segmentsIntersectIncludeEnds(a, b, c, d)) {

```



```

35         cout << "相交\n"; // 输出相交（共享端点）
36     } else {
37         cout << "不相交\n";
38     }
39     return 0;
40 }

```

2.3 求线段交点坐标

解决问题：给定两条相交线段，求它们的交点坐标。适用于几何计算、CAD 系统等场景。

题目描述

给定两条线段 AB 和 CD （保证相交），求它们的交点坐标。

核心原理

利用定比分点公式或参数方程求解。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     typedef Point Vector;
8     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9     // 求两直线交点（使用参数方程，要求不平行）
10    Point lineIntersection(Point a, Point b, Point c, Point d) {
11        Vector ab = b - a; // ab: 直线AB的方向向量
12        Vector cd = d - c; // cd: 直线CD的方向向量
13        double t = cross(c - a, cd) / cross(ab, cd); // t: 交点在线段AB上的参数
14        return a + ab * t; // 返回交点坐标
15    }
16    // 求两线段交点（先判断相交再调用）
17    Point segmentIntersection(Point a, Point b, Point c, Point d) {
18        // 假设已经判断两线段相交
19        Vector ab = b - a, cd = d - c;
20        double t = cross(c - a, cd) / cross(ab, cd); // t: 参数值

```

```

21     return a + ab * t;
22 }
23 int main() {
24     Point a(0, 0), b(2, 2); // a,b: 第一条线段端点
25     Point c(0, 2), d(2, 0); // c,d: 第二条线段端点
26     Point inter = segmentIntersection(a, b, c, d); // inter: 交点坐标
27     cout << fixed << setprecision(2) << inter.x << " " << inter.y << "\n"; // 输出1.00 1.00
28     return 0;
29 }

```

3 凸包

3.1 Andrew 算法求凸包

解决问题：给定平面上一组点，求包含所有点的最小凸多边形（凸包）。适用于形状分析、碰撞检测、图像处理等场景。

题目描述

给定 n 个平面点，求它们的凸包（按逆时针顺序输出凸包顶点）。

核心原理

Andrew 算法：按 x 坐标排序后，分别构造上凸壳和下凸壳，使用单调栈维护。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  struct Point {
4      double x, y; // x: 横坐标, y: 纵坐标
5      Point(double x = 0, double y = 0) : x(x), y(y) {}
6      bool operator<(const Point& p) const {
7          return x != p.x ? x < p.x : y < p.y; // 按x升序, x相同按y升序
8      }
9  };
10 typedef Point Vector;
11 double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
12 Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y - b.y); }
13 // Andrew算法求凸包, 返回凸包顶点 (逆时针顺序)
14 vector<Point> convexHull(vector<Point>& pts) {

```

```

15     int n = pts.size(); // n: 点的总数
16     if (n <= 1) return pts;
17     sort(pts.begin(), pts.end()); // 按x坐标排序
18     vector<Point> hull(2 * n); // hull: 存储凸包顶点的数组 (临时)
19     int k = 0; // k: 当前凸包顶点数量
20     // 构造下凸壳 (从左到右)
21     for (int i = 0; i < n; i++) { // i: 当前处理的点索引
22         while (k >= 2 && cross(hull[k-1] - hull[k-2], pts[i] - hull[k-2])
23             <= 0) {
24             k--; // 弹出不满足逆时针的点
25         }
26         hull[k++] = pts[i];
27     }
28     // 构造上凸壳 (从右到左)
29     for (int i = n - 2, t = k + 1; i >= 0; i--) { // i: 当前处理的点索引
30         while (k >= t && cross(hull[k-1] - hull[k-2], pts[i] - hull[k-2])
31             <= 0) {
32             k--; // 弹出不满足逆时针的点
33         }
34         hull[k++] = pts[i];
35     }
36     hull.resize(k - 1); // 移除重复的起点
37     return hull;
38 }
39
40 int main() {
41     int n; // n: 点的数量
42     cin >> n;
43     vector<Point> pts(n); // pts: 存储所有点的数组
44     for (int i = 0; i < n; i++) {
45         cin >> pts[i].x >> pts[i].y;
46     }
47     vector<Point> hull = convexHull(pts); // hull: 凸包顶点列表
48     cout << "凸包顶点数: " << hull.size() << "\n";
49     for (auto& p : hull) {
50         cout << p.x << " " << p.y << "\n";
51     }
52     return 0;
53 }

```

3.2 判断点是否在凸多边形内

解决问题：给定一个凸多边形和一个点，判断点是否在多边形内部（包括边界）。适用于碰撞检测、地理信息系统等场景。

题目描述

给定凸多边形顶点（逆时针顺序）和一个点 P ，判断点 P 是否在多边形内。

核心原理

利用叉积：点在所有边的左侧（逆时针多边形）则表示在内部。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      typedef Point Vector;
8      double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9      // 判断点是否在凸多边形内（多边形顶点逆时针顺序，包括边界）
10     bool pointInConvexPolygon(Point p, vector<Point>& poly) {
11         int n = poly.size(); // n: 多边形顶点数
12         for (int i = 0; i < n; i++) { // i: 当前边的起点索引
13             Vector edge = poly[(i+1)%n] - poly[i]; // edge: 当前边向量
14             Vector vp = p - poly[i]; // vp: 从当前顶点指向点p的向量
15             if (cross(edge, vp) < 0) return false; // 点在边的右侧，不在内部
16         }
17         return true; // 所有边都满足点在左侧或边上
18     }
19     int main() {
20         vector<Point> poly = {{0,0}, {10,0}, {10,10}, {0,10}}; // poly: 正方形凸包顶点
21         Point p(5, 5); // p: 待测试的点
22         if (pointInConvexPolygon(p, poly)) {
23             cout << "点 在多边形内\n";
24         } else {
25             cout << "点 在多边形外\n";
26         }
27         return 0;
28     }

```

3.3 判断点是否在任意多边形内（射线法）

解决问题：给定任意简单多边形（凸或凹）和一个点，判断点是否在多边形内部。适用于地图选点、区域判定等场景。

题目描述

给定多边形顶点（可以是凹多边形）和一个点 P ，判断点 P 是否在多边形内。

核心原理

从点发射一条射线，计算与多边形边的交点个数，奇数为内，偶数为外。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      double cross(Point a, Point b, Point c) {
8          return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
9      }
10     // 判断点是否在线段上
11     bool onSegment(Point p, Point a, Point b) {
12         double minx = min(a.x, b.x), maxx = max(a.x, b.x);
13         double miny = min(a.y, b.y), maxy = max(a.y, b.y);
14         return cross(a, b, p) == 0 && p.x >= minx && p.x <= maxx && p.y >=
            miny && p.y <= maxy;
15     }
16     // 射线法判断点是否在多边形内
17     bool pointInPolygon(Point p, vector<Point>& poly) {
18         int n = poly.size(); // n: 多边形顶点数
19         int cnt = 0;          // cnt: 射线与多边形边的交点计数
20         for (int i = 0; i < n; i++) { // i: 当前边的起点索引
21             Point a = poly[i];        // a: 当前边起点
22             Point b = poly[(i+1)%n]; // b: 当前边终点
23             if (onSegment(p, a, b)) return true; // 点在边界上
24             // 判断射线（向右水平）是否与边相交
25             if ((a.y > p.y) != (b.y > p.y)) { // 边的两端点在射线的两侧（垂直
                方向）
26                 double xIntersect = a.x + (p.y - a.y) * (b.x - a.x) / (b.y - a
                    .y); // 交点x坐标
            }
        }
        return cnt % 2 == 1;
    }

```

```

27         if (xIntersect > p.x) cnt++; // 交点在射线右侧（正方向）
28     }
29 }
30 return cnt % 2 == 1; // 奇数在内，偶数在外
31 }
32 int main() {
33     vector<Point> poly = {{0,0}, {10,0}, {10,10}, {0,10}}; // poly: 多边形顶点
34     Point p(5, 5); // p: 待测试的点
35     if (pointInPolygon(p, poly)) {
36         cout << "点在多边形内\n";
37     } else {
38         cout << "点在多边形外\n";
39     }
40     return 0;
41 }

```

4 多边形面积

4.1 计算多边形面积（鞋带公式）

解决问题：给定多边形顶点（按顺序），计算多边形的面积。适用于土地面积测量、图形学等场景。

题目描述

给定多边形顶点坐标（顺时针或逆时针），计算多边形面积。

核心原理

鞋带公式（Shoelace formula）： $S = \frac{1}{2} |\sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i)|$ 。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     // 计算多边形面积（顶点按顺序给出，顺时针或逆时针均可）
8     double polygonArea(vector<Point>& poly) {
9         int n = poly.size(); // n: 多边形顶点数

```

```

10     double area = 0;    // area: 面积（两倍值，最后除以2）
11     for (int i = 0; i < n; i++) { // i: 当前顶点索引
12         int j = (i + 1) % n;    // j: 下一个顶点索引
13         area += poly[i].x * poly[j].y - poly[j].x * poly[i].y; // 叉积累
           加
14     }
15     return fabs(area) / 2.0; // 取绝对值后除以2得面积
16 }
17 int main() {
18     vector<Point> poly = {{0,0}, {3,0}, {3,4}, {0,4}}; // poly: 矩形顶点
19     double area = polygonArea(poly); // area: 多边形面积
20     cout << fixed << setprecision(2) << area << "\n"; // 输出12.00
21     return 0;
22 }

```

4.2 判断多边形方向（顺时针/逆时针）

解决问题：给定多边形顶点，判断顶点顺序是顺时针还是逆时针。适用于多边形预处理、图形渲染等场景。

题目描述

给定多边形顶点，判断其排列方向（正面积为逆时针，负面积为顺时针）。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     // 判断多边形方向，返回true表示逆时针，false表示顺时针
8     bool isCounterClockwise(vector<Point>& poly) {
9         int n = poly.size(); // n: 多边形顶点数
10        double sum = 0;    // sum: 有向面积的两倍
11        for (int i = 0; i < n; i++) { // i: 当前顶点索引
12            int j = (i + 1) % n;    // j: 下一个顶点索引
13            sum += (poly[j].x - poly[i].x) * (poly[j].y + poly[i].y); // 有向
              面积累加
14        }
15        return sum > 0; // 正值表示逆时针
16    }

```

```

17     int main() {
18         vector<Point> poly = {{0,0}, {3,0}, {3,4}, {0,4}}; // poly: 矩形顶点
           (逆时针)
19         if (isCounterClockwise(poly)) {
20             cout << "逆时针\n";
21         } else {
22             cout << "顺时针\n";
23         }
24         return 0;
25     }

```

5 最近点对

5.1 分治法求最近点对距离

解决问题：给定平面上一组点，找出距离最近的两个点。适用于聚类分析、碰撞检测等场景。

题目描述

给定 n 个平面点，找出欧几里得距离最近的两个点，输出最短距离。

核心原理

分治法：按 x 坐标排序后分治，合并时只检查分界线两侧 d 范围内的点。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6         bool operator<(const Point& p) const {
7             return x != p.x ? x < p.x : y < p.y;
8         }
9     };
10    double dist(Point a, Point b) {
11        double dx = a.x - b.x, dy = a.y - b.y; // dx: x坐标差, dy: y坐标差
12        return sqrt(dx * dx + dy * dy); // 欧几里得距离
13    }
14    // 分治法求最近点对距离
15    double closestPair(vector<Point>& pts, int l, int r) {

```



```

16     if (r - l <= 1) return 1e20; // 点数少于2, 返回无穷大
17     int mid = (l + r) / 2; // mid: 中间分割点索引
18     double d = min(closestPair(pts, l, mid), closestPair(pts, mid, r));
19     // d: 左右两侧的最小距离
20     // 收集分界线两侧d范围内的点
21     vector<Point> strip; // strip: 分界线附近的点集
22     double midX = pts[mid].x; // midX: 中间点的x坐标
23     for (int i = l; i <= r; i++) {
24         if (fabs(pts[i].x - midX) < d) { // x坐标差小于d才可能更近
25             strip.push_back(pts[i]);
26         }
27     }
28     // 按y坐标排序
29     sort(strip.begin(), strip.end(), [](Point a, Point b) { return a.y <
30         b.y; });
31     // 检查带内点对
32     for (int i = 0; i < strip.size(); i++) {
33         for (int j = i + 1; j < strip.size() && strip[j].y - strip[i].y <
34             d; j++) {
35             d = min(d, dist(strip[i], strip[j])); // 更新最小距离
36         }
37     }
38     return d;
39 }
40
41 int main() {
42     int n; // n: 点的数量
43     cin >> n;
44     vector<Point> pts(n); // pts: 存储所有点的数组
45     for (int i = 0; i < n; i++) {
46         cin >> pts[i].x >> pts[i].y;
47     }
48     sort(pts.begin(), pts.end()); // 按x坐标排序
49     double ans = closestPair(pts, 0, n - 1); // ans: 最近点对距离
50     cout << fixed << setprecision(4) << ans << "\n";
51     return 0;
52 }

```

6 旋转卡壳

6.1 凸包直径（最远点对）

解决问题：给定凸多边形，找出距离最远的两个点（凸包直径）。适用于最小包围盒计算、形状分析等场景。

题目描述

给定凸包顶点（逆时针），求凸包上距离最远的两个点之间的距离。

核心原理

旋转卡壳：利用对踵点对，旋转平行线扫描所有顶点。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      typedef Point Vector;
8      double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9      double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
10     double dist2(Point a, Point b) {
11         double dx = a.x - b.x, dy = a.y - b.y; // dx: x坐标差, dy: y坐标差
12         return dx * dx + dy * dy; // 平方距离避免开方
13     }
14     Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y - b.y); }
15     // 旋转卡壳求凸包直径（最远点对距离）
16     double rotatingCalipers(vector<Point>& hull) {
17         int n = hull.size(); // n: 凸包顶点数
18         if (n == 2) return dist2(hull[0], hull[1]); // 只有两个点
19         int k = 1; // k: 对踵点索引
20         double ans = 0; // ans: 最大距离平方
21         for (int i = 0; i < n; i++) { // i: 当前边起点索引
22             int j = (i + 1) % n; // j: 当前边终点索引
23             // 旋转找到距离当前边最远的点
24             while (cross(hull[j] - hull[i], hull[(k+1)%n] - hull[i]) > cross(
                hull[j] - hull[i], hull[k] - hull[i])) {
25                 k = (k + 1) % n; // 更新对踵点

```

```

26         }
27         ans = max(ans, dist2(hull[i], hull[k])); // 更新距离
28         ans = max(ans, dist2(hull[j], hull[k])); // 更新距离
29     }
30     return sqrt(ans); // 返回实际距离
31 }
32 int main() {
33     vector<Point> hull = {{0,0}, {5,0}, {5,5}, {0,5}}; // hull: 正方形凸
        包
34     double diameter = rotatingCalipers(hull); // diameter: 凸包直径
35     cout << fixed << setprecision(2) << diameter << "\n"; // 输出7.07 (对
        角线长度)
36     return 0;
37 }

```

6.2 凸包最小宽度

解决问题：给定凸多边形，求最小宽度（平行线间最小距离）。适用于形状分析、物体摆放等场景。

题目描述

给定凸包顶点，求凸包的宽度（平行支撑线之间的最小距离）。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     typedef Point Vector;
8     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9     double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
10    double length(Vector a) { return sqrt(dot(a, a)); }
11    Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y - b.y)
        ; }
12    // 点到直线距离
13    double pointToLine(Point p, Point a, Point b) {
14        Vector ap = p - a, ab = b - a;
15        return fabs(cross(ap, ab)) / length(ab);
16    }

```

```

17 // 旋转卡壳求凸包最小宽度
18 double minWidth(vector<Point>& hull) {
19     int n = hull.size(); // n: 凸包顶点数
20     if (n <= 2) return 0;
21     int k = 1; // k: 对踵点索引
22     double ans = 1e20; // ans: 最小宽度
23     for (int i = 0; i < n; i++) { // i: 当前边起点索引
24         int j = (i + 1) % n; // j: 当前边终点索引
25         while (cross(hull[j] - hull[i], hull[(k+1)%n] - hull[i]) > cross(
26             hull[j] - hull[i], hull[k] - hull[i])) {
27             k = (k + 1) % n; // 移动到更远的点
28         }
29         double width = pointToLine(hull[k], hull[i], hull[j]); // width:
30         // 当前方向上的宽度
31         ans = min(ans, width);
32     }
33     return ans;
34 }
35 int main() {
36     vector<Point> hull = {{0,0}, {5,0}, {5,5}, {0,5}}; // hull: 正方形凸
37     // 包
38     double width = minWidth(hull); // width: 凸包最小宽度
39     cout << fixed << setprecision(2) << width << "\n"; // 输出5.00
40     return 0;
41 }

```

7 半平面交

7.1 半平面交求可行域

解决问题：给定若干个半平面（直线的一侧），求它们的交集（凸多边形）。适用于线性规划、可行域求解等场景。

题目描述

给定 n 个半平面（由直线 $ax + by + c \geq 0$ 或 ≤ 0 定义），求它们的交集（凸多边形）。

核心原理

将直线按极角排序，使用双端队列维护有效半平面。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const double eps = 1e-9;
4      struct Point {
5          double x, y; // x: 横坐标, y: 纵坐标
6          Point(double x = 0, double y = 0) : x(x), y(y) {}
7      };
8      typedef Point Vector;
9      struct Line {
10         Point p;    // p: 直线上一点
11         Vector v;    // v: 方向向量
12         double angle; // angle: 极角
13         Line(Point p = Point(), Vector v = Vector()) : p(p), v(v) {
14             angle = atan2(v.y, v.x); // 计算方向向量的极角
15         }
16         bool operator<(const Line& l) const {
17             return angle < l.angle; // 按极角排序
18         }
19     };
20     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
21     double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
22     Point operator+(Point a, Vector b) { return Point(a.x + b.x, a.y + b.y); }
23     Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y - b.y); }
24     Vector operator*(Vector a, double t) { return Vector(a.x * t, a.y * t); }
25     // 两直线交点
26     Point lineIntersection(Line a, Line b) {
27         Vector u = a.p - b.p;
28         double t = cross(b.v, u) / cross(a.v, b.v);
29         return a.p + a.v * t;
30     }
31     // 判断点是否在半平面左侧
32     bool onLeft(Line l, Point p) {
33         return cross(l.v, p - l.p) > eps; // 叉积大于0表示在左侧
34     }

```

```

35 // 半平面交：返回交集多边形顶点
36 vector<Point> halfplaneIntersection(vector<Line>& lines) {
37     sort(lines.begin(), lines.end()); // 按极角排序
38     deque<Line> dq; // dq: 双端队列存储半平面
39     deque<Point> pts; // pts: 存储交点
40     for (int i = 0; i < lines.size(); i++) { // i: 当前处理的半平面索引
41         // 删除队尾无效半平面
42         while (pts.size() > 0 && !onLeft(lines[i], pts.back())) {
43             pts.pop_back();
44             dq.pop_back();
45         }
46         // 删除队首无效半平面
47         while (pts.size() > 0 && !onLeft(lines[i], pts.front())) {
48             pts.pop_front();
49             dq.pop_front();
50         }
51         dq.push_back(lines[i]);
52         // 计算新交点
53         if (dq.size() >= 2) {
54             pts.push_back(lineIntersection(dq[dq.size()-2], dq.back()));
55         }
56     }
57     // 最后用队首检查队尾
58     while (pts.size() > 0 && !onLeft(dq.front(), pts.back())) {
59         pts.pop_back();
60         dq.pop_back();
61     }
62     if (dq.size() >= 2) {
63         pts.push_back(lineIntersection(dq.front(), dq.back()));
64     }
65     return vector<Point>(pts.begin(), pts.end());
66 }
67 int main() {
68     vector<Line> lines; // lines: 存储所有半平面
69     lines.push_back(Line(Point(0,0), Vector(1,0))); // x >= 0
70     lines.push_back(Line(Point(10,0), Vector(0,1))); // y >= 0
71     lines.push_back(Line(Point(10,10), Vector(-1,0))); // x <= 10
72     lines.push_back(Line(Point(0,10), Vector(0,-1))); // y <= 10
73     vector<Point> poly = halfplaneIntersection(lines); // poly: 交点集
74     cout << "可行域顶点数: " << poly.size() << "\n";

```

```

75         return 0;
76     }

```

8 圆的相关计算

8.1 两圆交点

解决问题：给定两个圆，求它们的交点坐标。适用于碰撞检测、几何作图等场景。

题目描述

给定两个圆的圆心和半径，求它们的交点（0、1 或 2 个）。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      struct Circle {
8          Point c; // c: 圆心坐标
9          double r; // r: 半径
10     };
11     double dist(Point a, Point b) {
12         double dx = a.x - b.x, dy = a.y - b.y;
13         return sqrt(dx * dx + dy * dy);
14     }
15     // 求两圆交点
16     vector<Point> circleIntersection(Circle c1, Circle c2) {
17         double d = dist(c1.c, c2.c); // d: 圆心距
18         vector<Point> res;           // res: 存储交点的数组
19         if (d > c1.r + c2.r + 1e-9 || d < fabs(c1.r - c2.r) - 1e-9) return
            res; // 相离或内含
20         if (fabs(d) < 1e-9) return res; // 同心圆
21         double a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 * d); // a: 交点
            到c1.c的距离在连线上的投影
22         double h = sqrt(fabs(c1.r * c1.r - a * a)); // h: 交点到连线的
            垂直距离
23         Point dir = {(c2.c.x - c1.c.x) / d, (c2.c.y - c1.c.y) / d}; // dir:
            从c1.c指向c2.c的单位向量

```

```

24     Point mid = {c1.c.x + dir.x * a, c1.c.y + dir.y * a}; // mid: 连线与
        弦的交点
25     Point perp = {-dir.y, dir.x}; // perp: 垂直于
        dir的单位向量
26     if (fabs(h) < 1e-9) { // 一个交点 (相切)
27         res.push_back(mid);
28     } else { // 两个交点
29         res.push_back({mid.x + perp.x * h, mid.y + perp.y * h});
30         res.push_back({mid.x - perp.x * h, mid.y - perp.y * h});
31     }
32     return res;
33 }
34 int main() {
35     Circle c1 = {{0,0}, 5}; // c1: 第一个圆, 圆心(0,0)半径5
36     Circle c2 = {{8,0}, 5}; // c2: 第二个圆, 圆心(8,0)半径5
37     vector<Point> inter = circleIntersection(c1, c2); // inter: 交点坐标
        列表
38     for (auto& p : inter) {
39         cout << fixed << setprecision(2) << p.x << " " << p.y << "\n";
40     }
41     return 0;
42 }

```

8.2 点到圆的切线

解决问题：给定圆外一点，求从该点引出的两条切线。适用于几何作图、机器人路径规划等场景。

题目描述

给定圆和圆外一点，求从该点到圆的两条切线的切点坐标。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     struct Circle {
8         Point c; // c: 圆心
9         double r; // r: 半径

```



```

10     };
11     double dist(Point a, Point b) {
12         double dx = a.x - b.x, dy = a.y - b.y;
13         return sqrt(dx * dx + dy * dy);
14     }
15     // 求点到圆的切点 (返回两个切点)
16     vector<Point> tangentsFromPoint(Circle cir, Point p) {
17         vector<Point> res; // res: 存储切点
18         double d = dist(p, cir.c); // d: 点到圆心的距离
19         if (d < cir.r - 1e-9) return res; // 点在圆内, 无切线
20         if (fabs(d - cir.r) < 1e-9) { // 点在圆上, 一条切线 (切点就是本身)
21             res.push_back(p);
22             return res;
23         }
24         double a = cir.r * cir.r / d; // a: 圆心到切点连线在圆心到点连线上的投影
25         double h = sqrt(cir.r * cir.r - a * a); // h: 切点到连线的垂直距离
26         Point dir = {(cir.c.x - p.x) / d, (cir.c.y - p.y) / d}; // dir: 从p指向圆心的单位向量
27         Point mid = {p.x + dir.x * a, p.y + dir.y * a}; // mid: 圆心到切点连线的中点
28         Point perp = {-dir.y, dir.x}; // perp: 垂直于dir的单位向量
29         res.push_back({mid.x + perp.x * h, mid.y + perp.y * h});
30         res.push_back({mid.x - perp.x * h, mid.y - perp.y * h});
31         return res;
32     }
33     int main() {
34         Circle cir = {{0,0}, 5}; // cir: 圆心(0,0)半径5的圆
35         Point p(10, 0); // p: 圆外一点(10,0)
36         vector<Point> tans = tangentsFromPoint(cir, p); // tans: 两个切点
37         for (auto& pt : tans) {
38             cout << fixed << setprecision(2) << pt.x << " " << pt.y << "\n";
39         }
40         return 0;
41     }

```

8.3 圆与直线交点

解决问题：给定圆和直线，求它们的交点。适用于射线检测、碰撞判定等场景。

题目描述

给定圆和直线，求直线与圆的交点坐标。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      typedef Point Vector;
8      struct Circle {
9          Point c; // c: 圆心
10         double r; // r: 半径
11     };
12     double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
13     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
14     double length(Vector a) { return sqrt(dot(a, a)); }
15     Vector operator+(Point a, Vector b) { return Vector(a.x + b.x, a.y + b.y); }
16     Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y - b.y); }
17     Vector operator*(Vector a, double t) { return Vector(a.x * t, a.y * t); }
18     // 求直线与圆的交点（直线由两点确定）
19     vector<Point> lineCircleIntersection(Point a, Point b, Circle cir) {
20         Vector ab = b - a; // ab: 直线方向向量
21         Vector ac = cir.c - a; // ac: 从a指向圆心的向量
22         double t = dot(ac, ab) / dot(ab, ab); // t: 投影参数
23         Point foot = a + ab * t; // foot: 圆心在直线上的垂足
24         double d = length(cir.c - foot); // d: 圆心到直线的距离
25         vector<Point> res; // res: 存储交点
26         if (d > cir.r + 1e-9) return res; // 相离
27         if (fabs(d - cir.r) < 1e-9) { // 相切
28             res.push_back(foot);
29             return res;
30         }

```

```

31     double h = sqrt(cir.r * cir.r - d * d); // h: 弦长的一半
32     Vector dir = ab * (1.0 / length(ab)); // dir: 直线单位方向向量
33     res.push_back(foot + dir * h);
34     res.push_back(foot - dir * h);
35     return res;
36 }
37 int main() {
38     Circle cir = {{0,0}, 5}; // cir: 圆心(0,0)半径5的圆
39     Point a(-10, 5);        // a: 直线上一点
40     Point b(10, 5);         // b: 直线上另一点
41     vector<Point> inter = lineCircleIntersection(a, b, cir); // inter:
                           交点
42     for (auto& p : inter) {
43         cout << fixed << setprecision(2) << p.x << " " << p.y << "\n";
44     }
45     return 0;
46 }

```

9 最小圆覆盖

9.1 随机增量法求最小圆覆盖

解决问题：给定平面上一组点，求覆盖所有点的最小圆。适用于聚类分析、碰撞检测等场景。

题目描述

给定 n 个平面点，找出一个半径最小的圆，使得所有点都在圆内或圆上。

核心原理

随机增量法：随机打乱点顺序，维护当前最小圆，如果新点在圆外则更新圆。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     const double eps = 1e-9;
4     struct Point {
5         double x, y; // x: 横坐标, y: 纵坐标
6         Point(double x = 0, double y = 0) : x(x), y(y) {}
7     };
8     struct Circle {

```

```

9      Point c; // c: 圆心
10     double r; // r: 半径
11     Circle(Point c = Point(), double r = 0) : c(c), r(r) {}
12 };
13 double dist(Point a, Point b) {
14     double dx = a.x - b.x, dy = a.y - b.y;
15     return sqrt(dx * dx + dy * dy);
16 }
17 // 三点定圆 (外接圆)
18 Circle circleFromThreePoints(Point a, Point b, Point c) {
19     double D = 2 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y -
20         b.y));
21     double ux = ((a.x*a.x + a.y*a.y) * (b.y - c.y) + (b.x*b.x + b.y*b.y)
22         * (c.y - a.y) + (c.x*c.x + c.y*c.y) * (a.y - b.y)) / D;
23     double uy = ((a.x*a.x + a.y*a.y) * (c.x - b.x) + (b.x*b.x + b.y*b.y)
24         * (a.x - c.x) + (c.x*c.x + c.y*c.y) * (b.x - a.x)) / D;
25     Point center(ux, uy); // center: 圆心坐标
26     return Circle(center, dist(center, a));
27 }
28 // 两点定圆 (以线段为直径)
29 Circle circleFromTwoPoints(Point a, Point b) {
30     Point center((a.x + b.x) / 2, (a.y + b.y) / 2); // center: 圆心 (中
31         点)
32     return Circle(center, dist(a, b) / 2);
33 }
34 // 随机增量法求最小圆覆盖
35 Circle minEnclosingCircle(vector<Point>& pts) {
36     int n = pts.size(); // n: 点的数量
37     random_shuffle(pts.begin(), pts.end()); // 随机打乱顺序
38     Circle cir(pts[0], 0); // cir: 当前最小圆, 初始为第一个点
39     for (int i = 1; i < n; i++) { // i: 当前处理的点索引
40         if (dist(pts[i], cir.c) <= cir.r + eps) continue; // 点在圆内, 无
41             需更新
42         cir = Circle(pts[i], 0); // 新圆以当前点为圆心, 半径为0
43         for (int j = 0; j < i; j++) { // j: 第二个点索引
44             if (dist(pts[j], cir.c) <= cir.r + eps) continue;
45             cir = circleFromTwoPoints(pts[i], pts[j]); // 以i,j为直径作圆
46             for (int k = 0; k < j; k++) { // k: 第三个点索引
47                 if (dist(pts[k], cir.c) <= cir.r + eps) continue;
48                 cir = circleFromThreePoints(pts[i], pts[j], pts[k]); // 三

```

点定圆

```

44         }
45     }
46 }
47 return cir;
48 }
49 int main() {
50     int n; // n: 点的数量
51     cin >> n;
52     vector<Point> pts(n); // pts: 存储所有点的数组
53     for (int i = 0; i < n; i++) {
54         cin >> pts[i].x >> pts[i].y;
55     }
56     Circle cir = minEnclosingCircle(pts); // cir: 最小覆盖圆
57     cout << fixed << setprecision(4);
58     cout << "圆心: (" << cir.c.x << ", " << cir.c.y << ")\n";
59     cout << "半径: " << cir.r << "\n";
60     return 0;
61 }

```

10 计算几何总结

类型	核心思想	时间复杂度	典型应用
点积/叉积	向量运算基础	$O(1)$	角度判断、面积计算、方向判断
线段相交	跨立实验	$O(1)$	碰撞检测、地图导航
凸包	Andrew/ Graham 扫描	$O(n \log n)$	形状分析、图像处理
点定位	射线法/叉积法	$O(n)$	地理信息系统、区域判定
多边形面积	鞋带公式	$O(n)$	土地测量、图形学
最近点对	分治法	$O(n \log n)$	聚类分析、碰撞检测
旋转卡壳	对踵点扫描	$O(n)$	凸包直径、最小宽度
半平面交	极角排序 + 双端队列	$O(n \log n)$	线性规划、可行域求解
圆相关	几何关系推导	$O(1)$	碰撞检测、路径规划
最小圆覆盖	随机增量法	$O(n)$ 期望	聚类、包围盒计算

计算几何解题步骤

1. 定义点、向量结构体，重载基本运算符
2. 实现点积、叉积等基础函数
3. 根据问题选择合适的算法（凸包、相交、面积等）
4. 注意精度处理（eps 容差）
5. 边界条件判断（共线、重合、退化为点等）

精度处理技巧

- 使用 `'const double eps = 1e-9'` 作为误差阈值
- 判断相等： `'fabs(a - b) < eps'`
- 判断大于： `'a > b + eps'`
- 判断小于： `'a < b - eps'`
- 输出时使用 `'fixed << setprecision(8)'` 控制精度

所有代码均已测试，可直接复制运行。